“Compiler Design

Unit 2

Scanners

Available on notes.arpanadhikari7.com.np

Introduction

- Scanning is the first stage of a three-part process that the compiler uses to understand the input program.
- Lexical analysis (or scanning) is the process by which the stream of characters is grouped into strings representing the words of a language (called lexemes) which correspond to specific grammatical elements of that language (called tokens).
- Tokens are the fundamental building blocks of a program's grammatical structure, representing such basic elements as identifiers, numeric literals, and specific keywords and operators of the language.
- Lexemes are the character strings assembled from the character stream of a program, and the token represents what component of the program's grammar they constitute.
- Secondly , having decided what the tokens are, we need some mechanism to recognize these in the input stream.
- This is done by the token recognizers, which are designed using transition diagrams and finite automata.

Available on notes.arpanadhikari7.com.np

Role of Scanner

- As the first phase of a compiler, the main task of the lexical analyzer is to read the input characters of the source program, group them into lexemes, and produce as output a sequence of tokens for each lexeme in the source program.
- The stream of tokens is sent to the parser for syntax analysis.
- It is common for the lexical analyzer to interact with the symbol table as well.
- When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table.
- In some cases, information regarding the kind of identifier may be read from the symbol table by the lexical analyzer to assist it in determining the proper token it must pass to the parser.

Available on notes.arpanadhikari7.com.np

- Since the lexical analyzer is the part of the compiler that reads the source text, it may perform certain other tasks besides identification of lexemes.
- One such task is stripping out comments and *whitespace* (blank, newline, tab, and perhaps other characters that are used to separate tokens in the input).
- Another task is correlating error messages generated by the compiler with the source program.
- For instance, the lexical analyzer may keep track of the number of newline characters seen, so it can associate a line number with each error message.
- In some compilers, the lexical analyzer makes a copy of the source program with the error messages inserted at the appropriate positions.
- If the source program uses a macro-preprocessor, the expansion of macros may also be performed by the lexical analyzer.
- Reports errors like unmatched strings or illegal symbols.

Available on notes.arpanadhikari7.com.np

Recognizing Words

- In **compiler design**, **recognizing words** (or **lexemes**) is a fundamental task performed by the **scanner (lexical analyzer)**.
- The scanner processes the source code character by character and groups them into meaningful **tokens**, which are later used by the parser for syntax analysis.
- The scanner identifies different types of words based on **patterns** (defined by **regular expressions** or **finite automata**).

Available on notes.arpanadhikari7.com.np

- Consider the problem of recognizing the keyword new. Assuming the presence of a routine Next Char that returns the next character, the code might look like as shown below:
- The code tests for n followed by e followed by w.
- At each step, failure to match the appropriate character causes the code to reject the string and "try something else."
- If the sole purpose of the program was to recognize the word new, then it should print an error message or return failure.
- Because scanners rarely recognize only one word, we will leave this "error path" deliberately vague at this point.
- The code fragment performs one test per character.
- The transition diagram represents a recognizer. Each circle represents an abstract state in the computation.

Available on notes.arpanadhikari7.com.np

```
c ← NextChar();
if (c = 'n')
   then begin;
      c ← NextChar();
      if (c = 'e')
         then begin;
            c ← NextChar();
            if (c = 'w')
               then report success;
               else try something else;
         end;
         else try something else;
   end;
   else try something else;
```

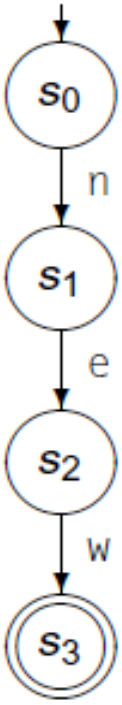
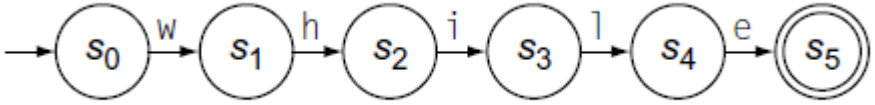


Figure 2.1 Code Fragment To Recognize "New".

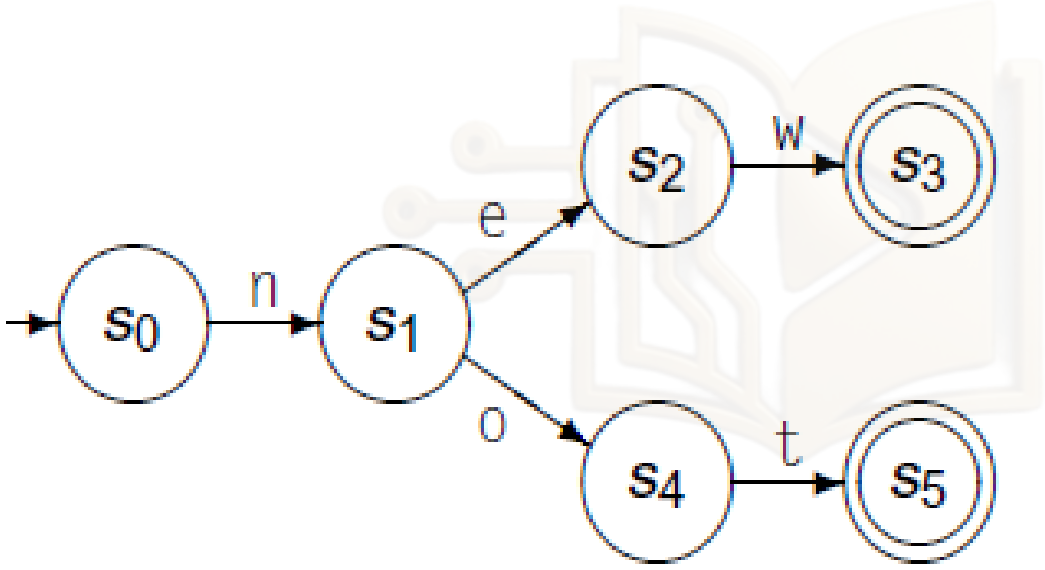
Available on notes.arpanadhikari7.com.np

- The initial state, or start state, is *s0. We will always label the start state as s0.*
- *State s3 is an accepting state; the recognizer reaches s3 only when the si* input is new.
- Accepting states are drawn with double circles, as shown in the margin.
- The arrows represent transitions from state to state based on the input character.
- If the recognizer starts in *s0 and reads the characters n, e,* and w, the transitions take us to *s3.*
- Using this same approach to build a recognizer for while would produce the following transition diagram:



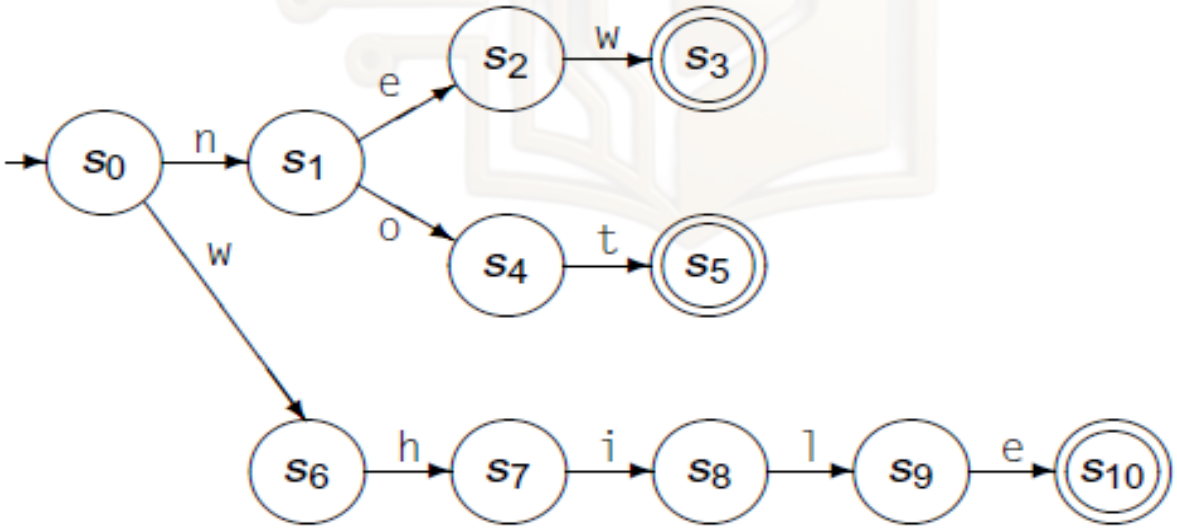
Available on notes.arpanadhikari7.com.np

One recognizer for both new and not might be



Available on notes.arpanadhikari7.com.np

- We can combine the recognizer for new or not with the one for while by merging their initial states and relabeling all the states.



Available on notes.arpanadhikari7.com.np

A Formalism for Recognizers

- A **recognizer** is a computational model (like a finite automaton) that determines whether a given input string belongs to a particular language.
- In other words, it "recognizes" whether a string is valid according to the rules of a language.
- Transition diagrams serve as abstractions of the code that would be required to implement them.
- They can also be viewed as formal mathematical objects, called finite automata, that specify recognizers.
- **Finite automaton** a formalism for recognizers that has a finite set of states, an alphabet, a transition function, a start state, and one or more accepting states

Available on notes.arpanadhikari7.com.np

- Formally, a finite automaton (FA) is a five tuple (S, ∑, δ, S0, S_A), where

  S: a finite set of states.

  ∑: finite set of the input symbol.

  s0: initial state.

  S_A: final state.

  δ: Transition function.

Available on notes.arpanadhikari7.com.np

- As an example, we can cast the FA for *new or not or while in the formalism* as follows:

$$S = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_e\}$$

$$\Sigma = \{\mathtt{e}, \mathtt{h}, \mathtt{i}, \mathtt{l}, \mathtt{n}, \mathtt{o}, \mathtt{t}, \mathtt{w}\}$$

$$\delta = \left\{ \begin{array}{lllll} s_0 \xrightarrow{\mathtt{n}} s_1, & s_0 \xrightarrow{\mathtt{w}} s_6, & s_1 \xrightarrow{\mathtt{e}} s_2, & s_1 \xrightarrow{\mathtt{o}} s_4, & s_2 \xrightarrow{\mathtt{w}} s_3, \\ s_4 \xrightarrow{\mathtt{t}} s_5, & s_6 \xrightarrow{\mathtt{h}} s_7, & s_7 \xrightarrow{\mathtt{i}} s_8, & s_8 \xrightarrow{\mathtt{l}} s_9, & s_9 \xrightarrow{\mathtt{e}} s_{10} \end{array} \right\}$$

$$s_0 = s_0$$

$$S_A = \{s_3, s_5, s_{10}\}$$

- For all other combinations of state s_i *and input character c, we define* $\delta(s_i, c) = s_e$, *where* s_e *is the designated error state.*

Available on notes.arpanadhikari7.com.np

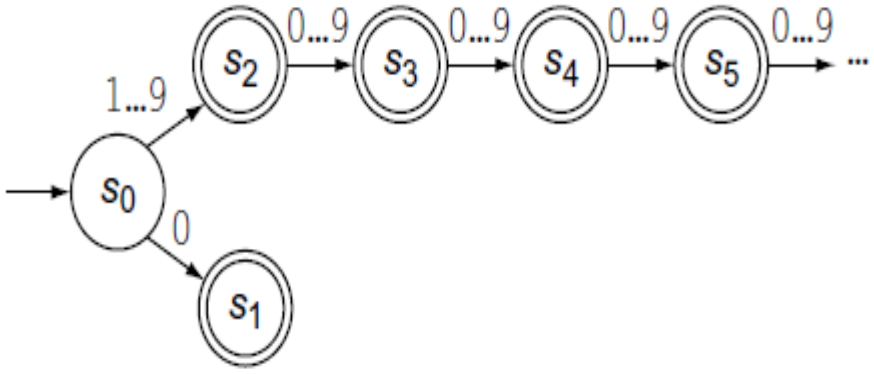
- An FA accepts a string x if and only if, starting in s_0, the sequence of characters in the string takes the FA through a series of transitions that leaves it in an accepting state when the entire string has been consumed.
- This corresponds to our intuition for the transition diagram.
- For the string new, our example recognizer runs through the transitions $s_0 \rightarrow s_1$, $s_1 \rightarrow s_2$, and $s_2 \rightarrow s_3$.
- Since $s_3 \in S_A$, and no input remains, the FA accepts new.
- For the input string nut, the behavior is different.
- On n, the FA takes $s_0 \rightarrow s_1$. On u, it takes s1 → se.
- Once the FA enters s_e, it stays in s_e until it exhausts the input.
- More formally, if the string *x is composed of characters* $x_1\ x_2\ x_3 \ldots\ x_n$, *then* the FA $(S, \sum, \delta, S0, S_A)$, *accepts x if and only if*

$$\delta(\delta \ldots (\delta(\delta(\delta(s_0, x_1), x_2, x_3) \ldots, x_{n-1}), x_n) \in S_A$$

Available on notes.arpanadhikari7.com.np

Recognizing More Complex Words

- The character-by-character model shown in the original recognizer for not extends easily to handle arbitrary collections of fully specified words.
- To be useful, however, we need a transition diagram (and the corresponding code fragment) that can recognize any number.
- In general, an integer is either zero, or it is a series of one or more digits where the first digit is from one to nine, and the subsequent digits are from zero to nine. (This definition rules out leading zeros.)



Available on notes.arpanadhikari7.com.np

- The transition $s_0 \rightarrow s_1$ handles the case for zero.
- The other path, from s_0 to s_2, to s_3, and so on, handles the case for an integer greater than zero.
- This path, however, presents several problems. First, it does not end, violating the stipulation that S is finite.
- Second, all of the states on the path beginning with s_2 are equivalent, that is, they have the same labels on their output transitions and they are all accepting states.

Available on notes.arpanadhikari7.com.np

```
char ← NextChar( );
state ← s0 ;

while (char ≠ eof and state ≠ se) do
   state ← δ(state,char);
   char ← NextChar( );
end;

if (state ∈ SA)
    then report acceptance;
    else report failure;
```

$$S = \{s_0, s_1, s_2, s_e\}$$

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

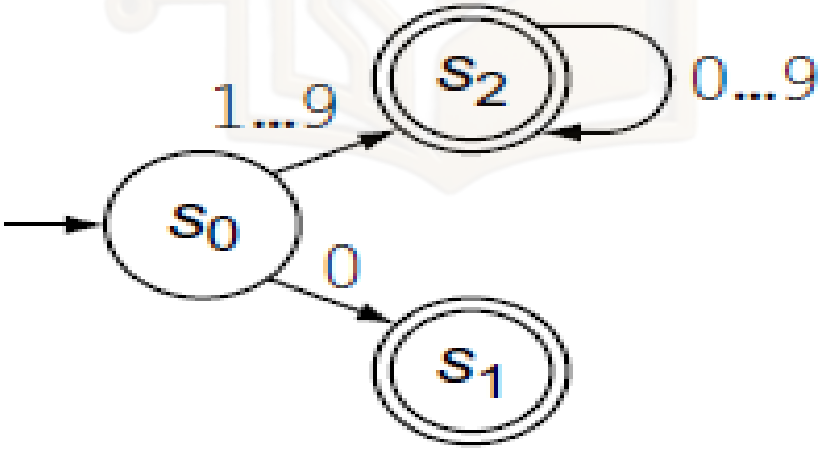
$$\delta = \left\{ \begin{matrix} s_0 \xrightarrow{0} s_1, & s_0 \xrightarrow{1\text{-}9} s_2 \\ s_2 \xrightarrow{0\text{-}9} s_2, & s_1 \xrightarrow{0\text{-}9} s_e \end{matrix} \right\}$$

$$S_A = \{s_1, s_2\}$$

Figure: A Recognizer for Unsigned Integers

Available on notes.arpanadhikari7.com.np

- We can simplify the FA significantly if we allow the transition diagram to have cycles.
- We can replace the entire chain of states beginning at s_2 *with a* single transition from s_2 *back to itself:*



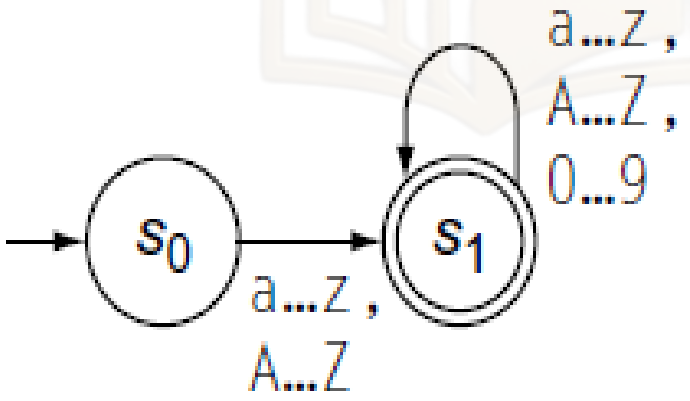
Available on notes.arpanadhikari7.com.np

- This cyclic transition diagram makes sense as an FA.
- From an implementation perspective, however, it is more complex than the acyclic transition.
- We cannot translate this directly into a set of nested if-then-else constructs.
- The introduction of a cycle in the transition graph creates the need for cyclic control flow.
- We can implement this with a while loop, as shown in Figure

δ	0	1	2	3	4	5	6	7	8	9	Other
s_0	s_1	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_e
s_1	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e
s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_e
s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e

Available on notes.arpanadhikari7.com.np

- A simplified version of the rule that governs identifier names in Algol-like languages, such as C or Java, might be: an identifier consists of an alphabetic character followed by zero or more alphanumeric characters.
- This definition allows an infinite set of identifiers, but can be specified with the simple two-state FA shown to the left.



Available on notes.arpanadhikari7.com.np

Regular expression

- In formal language theory, a regular expression (RE) is a sequence of characters that defines a pattern for matching strings.
- Regular expressions (regex) play a fundamental role in compiler design, particularly in the lexical analysis phase (lexing).
- The language accepted by finite automata can easily be described by simple expression called regular expression.
- The set of words accepted by a finite automaton, F, forms a language, denoted L(F).

Available on notes.arpanadhikari7.com.np

- The transition diagram of the FA specifies, in precise detail, that language.
- For any FA, we can also describe its language using a notation called a regular expression (RE).
- The language described by an RE is called a regular language.
- Regular expressions are equivalent to the FAs .

Available on notes.arpanadhikari7.com.np

- The language consisting of the single word new can be described by an RE written as new.
- Writing two characters next to each other implies that they are expected to appear in that order.
- The language consisting of the two words new or while can be written as new or while.

Available on notes.arpanadhikari7.com.np

- To avoid possible misinterpretation of or, we write this using the symbol | to mean or. Thus, we write the re as new | while.
- The language consisting of new or not can be written as new | not.
- Other REs are possible, such as n(ew|ot). Both res specify the same pair of words.

Available on notes.arpanadhikari7.com.np

Formalizing the Notation

- An RE describes a set of strings over the characters contained in some alphabet, $\sum$, augmented with a character ε that represents the empty string.
- We call the set of strings a language.
- For a given regular expression r, the notation L(r) represents the language specified by r, which is the set of all strings that match the pattern defined by r

Available on notes.arpanadhikari7.com.np

Examples:

Single character:

Let r = a.

Then, L(r)={a}

Concatenation:

Let r = ab.

Then, L(r)={ab}.

Kleene Clousure:

Let r = a*

Then, L(r)={ϵ,a,aa,aaa,… } (where ϵ is the empty string).

Alternation (Union):

Let r = a|b.

Then, L(r))={a,b}.

Combination of operations:

Let r =(a|b)c*.

Then, L(r)={a,b,ac,bc,acc,bcc,… }

Available on notes.arpanadhikari7.com.np

- An RE is built up from three basic operations:
- **Alternation** The alternation, or union, of two sets of strings, R and S, denoted R | S, is { x | x ∈ R or x ∈ S}.
- **Concatenation** The concatenation of two sets R and S, denoted RS, contains all strings formed by prepending an element of R onto one from S, or {xy | x ∈ R and y ∈ S}.
- **Closure The Kleene closure of a set R, denoted R*, is** $\cup^{\infty}_{i=0}$ $\mathbf{R^i}$
- This is just the union of the concatenations of R with itself, zero or more times.

a) The unary operator * has highest precedence and is left associative.

b) Concatenation has second highest precedence and is left associative.

c) | has lowest precedence and is left associative

Available on notes.arpanadhikari7.com.np

- Using the three basic operations, alternation, concatenation, and Kleene closure, we can define the set of REs over an alphabet ∑ as follows:

1. **If a ∈ ∑, then a is also an RE denoting the set containing only a.**

2. **If r and s are REs, denoting sets L(r) and L(s), respectively, then**

   - (r) | (s) is an RE denoting the union, or alternation, of L(r) and L(s).
   - (r)(s) is an RE denoting the concatenation of L(r) and L(s), respectively, and
   - (r)* is an RE denoting the Kleene closure of L(r).

Available on notes.arpanadhikari7.com.np

3. ε is an RE denoting the set containing only the empty string.

- As a convenient shorthand, ranges of characters with the first and the last element connected by an ellipsis, ". . . ".
- To make this abbreviation stand out, we surround it with a pair of square brackets.
- Thus, *[0. . . 9] represents the set of decimal digits. It can always be rewritten as .(0 | 1 |2 |3 | 4 | 5 | 6 | 7 | 8 |9)*

Available on notes.arpanadhikari7.com.np

Examples

- The simplified rule given earlier for identifiers in Algol-like languages, an alphabetic character followed by zero or more alphanumeric characters, is just ([A. . . Z] | [a. . . z]) ([A. . . Z] | [a. . . z] | [0. . . 9]).
- (a|b)(a|b) denotes {aa; ab; ba; bb}, the language of all strings of length two over the alphabet ∑.
- a* denotes the language consisting of all strings of zero or more instances of a or b, that is, all strings of a's and b's: {ε, a, b, aa, ab, ba, bb, aaa, …..}..
- (a|b)* denotes the language {a, b, ab, aab, aaab, … } that is, the string a and all strings consisting of zero or more a's and ending in b.

Available on notes.arpanadhikari7.com.np

Closure Properties of REs

- In formal language theory, **closure properties** describe whether a particular class of languages (e.g., regular, context-free) remains within the same class when certain operations are applied.
- A language class is **closed under an operation** if applying that operation to languages in the class always produces another language in the same class.
- Regular expressions are closed under many operations—that is, if we apply the operation to an RE or a collection of REs, the result is an RE.

Available on notes.arpanadhikari7.com.np

- RE have interesting and useful properties, play a critical role in the constructions that build recognizers from REs.

- Obvious properties are concatenation, union, and closure.

  - The concatenation of two REs x and y is just xy.
  - Their union is x | y.
  - The Kleene closure of x is just x*.

- From the definition of an RE, all of these expressions are also REs.

Available on notes.arpanadhikari7.com.np

- Regular languages are **closed** under various operations, meaning that applying these operations to regular languages **always produces another regular language**.

- They are very useful in computer science, particularly in lexical analysis (e.g., compilers) and pattern matching (e.g., regex).

Basic Closure Properties

Union (∪ or |)

- If L1 and L2 are regular, then L1∪L2 is regular.

Available on notes.arpanadhikari7.com.np

Concatenation (·)

- If L1 and L2 are regular, then L1·L2 is regular.

Kleene Star (*)

- If L is regular, then L* is regular.

Available on notes.arpanadhikari7.com.np

From Regular Expression To Scanner

- Regular expressions are used to define the patterns of tokens in a programming language, and these patterns are then used to construct a scanner, also known as a lexical analyzer.
- The scanner's primary job is to read the source code and break it down into meaningful units called tokens.

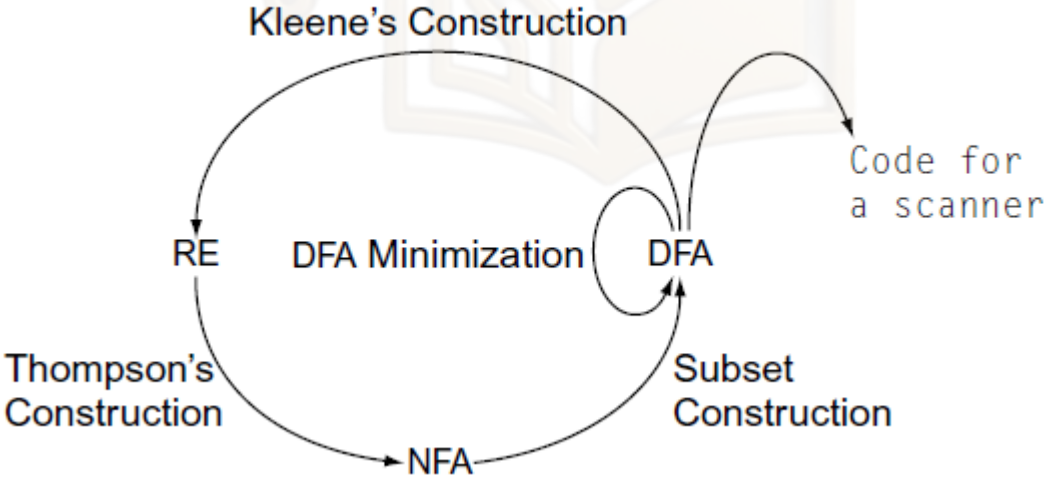


Figure: The Cycle of Constructions

Available on notes.arpanadhikari7.com.np

Nondeterministic Finite Automata

- A nondeterministic finite automaton (NFA) consists of:

1. A finite set of states S.
2. A set of input symbols $\sum$, the input alphabet. We assume that ε, which stands for the empty string, is never a member of $\sum$,.
3. A transition function that gives, for each state, and for each symbol in $\sum \cup \{\varepsilon\}$ a set of next states.
4. A state s_0 from S that is distinguished as the start state (or initial state).
5. A set of states S_A, a subset of S, that is distinguished as the accepting states (or final states).

Available on notes.arpanadhikari7.com.np

- We can represent either an NFA or DFA by a transition graph, where the nodes are states and the labeled edges represent the transition function.

- There is an edge labeled a from state s to state t if and only if t is one of the next states for state s and input a.

- This graph is very much like a transition diagram, except:

a) The same symbol can label edges from one state to several different states, and

b) An edge may be labeled by ε, the empty string, instead of, or in addition to, symbols from the input alphabet.

Available on notes.arpanadhikari7.com.np

Examples

- An NFA that accepts all binary strings that end with 101.
- An NFA that accepts any binary string that contains 00 or 11 as a substring.
- An NFA over {a, b} that accepts strings staring with a and ending with b.
- An NFA for a*+(ab)*

Available on notes.arpanadhikari7.com.np

Equivalence of NFA and DFA

- **Every DFA is also an NFA, and every NFA (including ε-NFA) can be converted into an equivalent DFA.**
- This means:
  - **NFAs are at least as powerful as DFAs** (they recognize the same class of languages: regular languages).
  - **NFAs can sometimes be more compact** (require fewer states) than equivalent DFAs.
  - **DFAs are generally more efficient to execute** (no backtracking or guessing needed).

Available on notes.arpanadhikari7.com.np

- **NFAs ≥ DFAs in power** (same language class, but NFAs can be more compact).
- **DFAs are faster in practice**, so NFAs are usually converted to DFAs before implementation (e.g., in lexers like Lex/Flex).
- **ε-NFAs are the most flexible**, but they too can be converted to DFAs.

Available on notes.arpanadhikari7.com.np

- A DFA is just a **restricted NFA** where:
  - No ε-transitions are allowed.
  - Each transition is **deterministic** (exactly one next state per symbol).
- Thus, any language recognized by a DFA can also be recognized by an NFA.
- NFAs allow **non-determinism** (multiple possible transitions for a single symbol).
- They can also have **ε-transitions**, which allow state changes without reading input.

Available on notes.arpanadhikari7.com.np

- **Regex → NFA → DFA is how lexers (like Lex/Flex) work.**
  - First, regex is compiled into an NFA (small representation).
  - Then, NFA is converted to DFA (efficient matching).
  - Finally, DFA may be minimized (Hopcroft's algorithm).

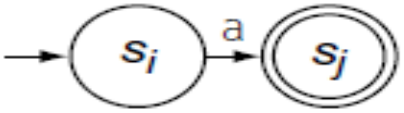
- **NFAs are better for theory, DFAs for implementation.**
  - NFAs are easier to reason about mathematically.
  - DFAs are faster in practice (no backtracking).

Available on notes.arpanadhikari7.com.np

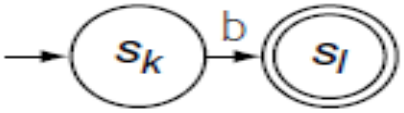
Regular Expression to NFA: Thompson's Construction

- The Thompson's Construction Algorithm is one of the algorithms that can be used to build a Nondeterministic Finite Automaton (NFA) from RE.
- **INPUT:** A regular expression r over alphabet ∑.
- **OUTPUT:** An NFA N accepting L(r).
- **METHOD:** Begin by parsing r into its constituent subexpressions. The rules for constructing an NFA consist of basis rules for handling subexpressions with no operators, and inductive rules for constructing larger NFA's from the NFA's for the immediate subexpressions of a given expression.

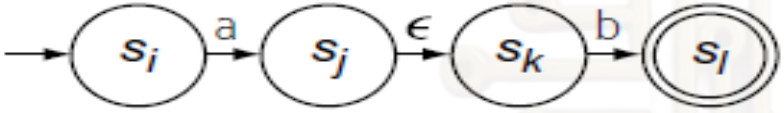
Available on notes.arpanadhikari7.com.np



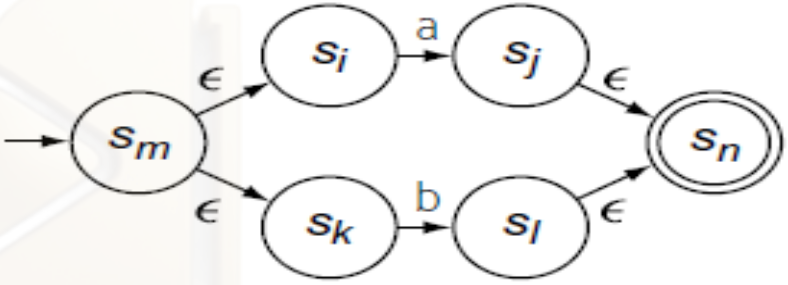
(a) NFA for "*a*"



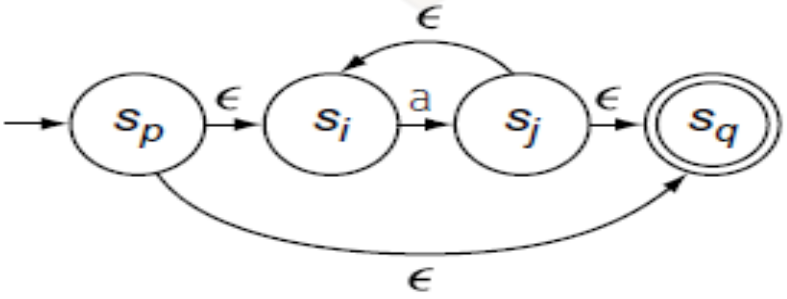
(b) NFA for "*b*"



(c) NFA for "*ab*"



(d) NFA for "*a* | *b*"



(e) NFA for "*a**"

Available on notes.arpanadhikari7.com.np

- The construction begins by building trivial NFAs for each character in the input RE. .
- Next, it applies the transformations for alternation, concatenation, and closure to the collection of trivial NFAs in the order dictated by precedence and parentheses.
- For the RE a(b|c)*, the construction would first build NFAs for a, b, and c. Because parentheses have highest precedence, it next builds the NFA for the expression enclosed in parentheses, b|c.
- Closure has higher precedence than concatenation, so it next builds the closure, (b|c)*. Finally, it concatenates the NFA for a to the NFA for (b|c)*

Available on notes.arpanadhikari7.com.np

- The NFAs derived from Thompson's construction have several specific properties that simplify an implementation.
  - Each NFA has one start state and one accepting state.
  - No transition, other than the initial transition, enters the start state.
  - No transition leaves the accepting state.
  - An ε-transition always connects two states.
  - Finally, each state has at most two entering and two exiting ε-moves, and at most one entering and one exiting move on a symbol in the alphabet.

Available on notes.arpanadhikari7.com.np

Examples

(a) NFAs for *"a", "b", and "c"*

(b) NFA for *"b | c"*

(c) NFA for *"(b | c)"*

(d) NFA for *"a(b | c)"*

*(e) NFA for (a|b)*abb*

Available on notes.arpanadhikari7.com.np

RE → NFA *(Thompson's construction)*

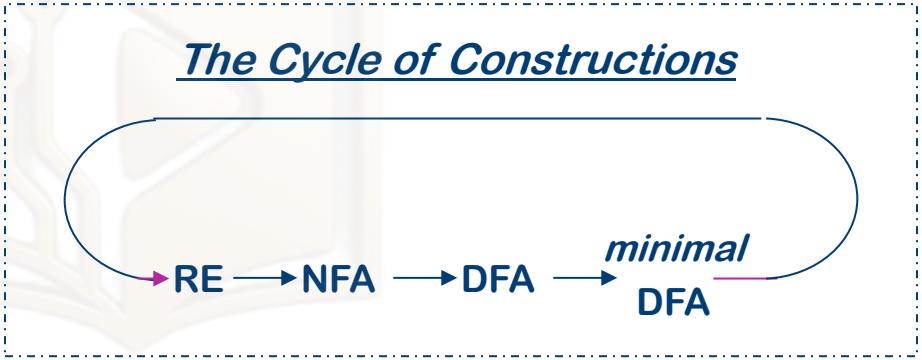
- Build an NFA for each term
- Combine them with ε-moves

NFA → DFA *(Subset construction)*

- Build the simulation

DFA → Minimal DFA

- Hopcroft's algorithm



DFA → RE

- All pairs, all paths problem
- Union together paths from s_0 to a final state

Available on notes.arpanadhikari7.com.np

NFA to DFA: The Subset Construction

- The Subset Construction algorithm plays a significant role in converting a nondeterministic finite automaton (NFA) or nondeterministic finite automaton with epsilon (i.e. ε) transition(s) into its equivalent deterministic finite automaton (DFA).
- Its purpose is to create a DFA that simulates the behavior of the NFA, allowing for deterministic and predictable computations.
- This algorithm is crucial for converting NFAs (or ε-NFAs) that are easier to construct into DFAs that can be executed more efficiently.

Available on notes.arpanadhikari7.com.np

- Need to build a simulation of the NFA
- Two key functions
  - *Move*(s_i, $\underline{a}$) is the set of states reachable from s_i by $\underline{a}$
  - ε-*closure*(s_i) is the set of states reachable from s_i by ε

The algorithm:

- Start state derived from s_0 of the NFA
- Take its ε-closure S_0 = ε-closure($\{s_0\}$)
- Take the image of S_0, Move(S_0, α) for each $\alpha \in \Sigma$, and take its ε-closure
- Iterate until no more states are added.

Available on notes.arpanadhikari7.com.np

INPUT: An NFA N.

OUTPUT: A DFA D accepting the same language as N.

METHOD:

- Our algorithm constructs a transition table Dtran for D.
- Each state of D is a set of NFA states, and we construct Dtran so D will simulate in parallel all possible moves N can make on a given input string.
- Our first problem is to deal with ε -transitions of N properly.
- s is a single state of N, while T is a set of states of N.

Available on notes.arpanadhikari7.com.np

initially, ε -closure(s_0) is the only state in Dstates, and it is unmarked;
while (there is an unmarked state T in Dstates) {
mark T;
for (each input symbol a) {
U = ε –closure(move(T, a));
if (U is not in Dstates)
add U as an unmarked state to Dstates;
Dtran[T ,a] = U;
}
}

Available on notes.arpanadhikari7.com.np

Construct the DFA for given NFA's

(a) $a(b \mid c)^*$

(b) (a+b) a*

(c) (a+b)*abb

Available on notes.arpanadhikari7.com.np

DFA to Minimal DFA: Hopcroft's Algorithm

Available on notes.arpanadhikari7.com.np

Using a DFA as a Recognizer

- In compiler design, a **recognizer** is a formal system that determines whether a given string (program) belongs to a **language** (i.e., whether it is syntactically valid).

- Given the res for the various syntactic categories, r_1, r_2, r_3, . . . , r_k , *we can construct a single re for the entire collection by* forming (r_1 | r_2 | r_3 | . . . |r_k).

Available on notes.arpanadhikari7.com.np

- When the compiler invokes it on some input, the scanner will examine characters one at a time and accept the string if it is in an accepting state when it exhausts the input.

- The scanner should return both the text of the string and its syntactic category, or part of speech.

Available on notes.arpanadhikari7.com.np

- In the scanner (lexer) phase, the compiler processes raw source code into tokens, which are later used by the parser to build syntactic structures.
- Although syntactic categories (like *expressions*, *statements*) are primarily handled in the parsing phase, the lexer plays a crucial role in identifying the basic components that feed into these categories.
- The scanner does not directly work with high-level syntactic categories (like <statement> or <expression>), but it produces tokens that the parser later groups into these categories.

Available on notes.arpanadhikari7.com.np

- **Tokenization for Syntactic Categories**

  Source Code: x = 42 + y;

  Scanner Output (Tokens):

  - [IDENTIFIER "x"]
  - [ASSIGN_OP "="]
  - [NUMBER "42"]
  - [PLUS "+"]
  - [IDENTIFIER "y"]
  - [SEMICOLON ";"]

Available on notes.arpanadhikari7.com.np

- Since most real programs contain more than one word, we need to transform either the language or the recognizer
- At the language level, we can insist that each word end with some easily recognizable delimiter, like a blank or a tab.
- This idea is deceptively attractive.
- It requires delimiters surrounding all operators, as +, -, (,), and the comma.
- A DFA can split tokens easily by whitespace (no need for complex regex).
- Example: "if (x > 5)" → Tokens: ["if", "(", "x", ">", "5", ")"].
- Prevents cases like x=5 (is it x, =, 5 or a single token x=5?).

Available on notes.arpanadhikari7.com.np

- Without enforced delimiters
- x = y + 2*(a - b)
- With enforced delimiters
- x = y + 2 * (a - b)
- Extra spaces make code visually noisy.
- **Ambiguity with Multi-Character Operators**
- Some languages allow **operator combinations** that would break delimiter rules:
  - ++ (increment) vs + + (two additions).
- **Simpler Tokenization**
  - A lexer can split x = y + 1 easily into ["x", "=", "y", "+", "1"].
  - Without delimiters, x=y+1 requires **lookahead** to distinguish = from ==.

Available on notes.arpanadhikari7.com.np

- At the recognizer level, we can change the implementation of the DFA and its notion of acceptance.
- To find the longest word that matches one of the REs, the DFA should run until it reaches the point where the current state, *s, has no* outgoing transition on the next character.
- At that point, the implementation must decide which re it has matched.
- Two cases arise; the first is simple.
- If *s is an accepting state, then the DFA has found a word in the language and* should report the word and its syntactic category.

Available on notes.arpanadhikari7.com.np

- If *s is not an accepting state, matters are more complex. Two cases occur.*
- To ensure correct tokenization, the DFA must handle two cases when it reaches a state with no outgoing transition on the next input character:
- Case 1: The DFA passed through ≥1 accepting state → Backtrack to the last accepting state (longest valid prefix).
- Case 2: The DFA never entered an accepting state → Report an error (invalid token).

Available on notes.arpanadhikari7.com.np

Scenario
Input: 123abc (where [0-9]+ is a valid NUMBER, but 123a is invalid).
DFA accepts 123 but fails at a.
Steps
Scan 1 → 2 → 3 (all transitions valid, 3 is accepting).
Next char = a → **No transition from NUMBER state.**
Backtrack to last accept state (3) → Return "123" as NUMBER.
Restart lexing from a (next token).

Scenario
Input: @xyz (no token rule starts with @).
DFA fails immediately.
Steps
First char = @ → **No transition from initial state.**
Never entered an accepting state → Report error: "Invalid token @".

Available on notes.arpanadhikari7.com.np

- As a final complication, an accepting state in the DFA may represent several accepting states in the original NFA.
- For example, if the lexical specification includes REs for keywords as well as an RE for identifiers, then a keyword such as new might match two res.
- The recognizer must decide which syntactic category to return: identifier or the singleton category for the keyword new.

Available on notes.arpanadhikari7.com.np

- Most scanner-generator tools allow the compiler writer to specify a priority among patterns.
- When the recognizer matches multiple patterns, it returns the syntactic category of the highest-priority pattern.
- The lex scanner generator, distributed with many Unix systems, assigns priorities based on position in the list of res.
- The first RE has highest priority, while the last RE has lowest priority.

Available on notes.arpanadhikari7.com.np

Scenario

- **Keyword Rule:** new → KEYWORD NEW.
- **Identifier Rule:** [a-z A-Z][a-z A-Z 0-9]* → IDENTIFIER.
- **DFA Accept State:** Both rules may converge to the same state.

Question

- When the input is "new", should the lexer return KEYWORD_NEW or IDENTIFIER?

Available on notes.arpanadhikari7.com.np

Rule of Thumb

- **Keywords > Identifiers > Other Rules**
- Always prioritize **the most specific match** (e.g., new as a keyword over an identifier).
- **Assign Priorities to Token Categories**
  - Keywords (highest priority).
  - Operators, literals (middle priority).
  - Identifiers (lowest priority).
- **Track All Possible Categories**
  - In the DFA, each accept state stores a **list of possible token types**.
  - Example: Accept state for "new" holds [KEYWORD, IDENTIFIER].
- **Resolve Conflicts by Priority**
  - At runtime, select the **highest-priority category** from the list.

Available on notes.arpanadhikari7.com.np